### Meta Squared

Singleton classes are classes, and classes are objects, and objects have singleton classes.... Can you see where this train of thought is going? Like any other object, a singleton class must have its own singleton class:

```
class << "abc"
  class << self
    self  # => #<Class:#<Class:#<String:0x33552c>>>
  end
end
```

If you ever find a practical use for singleton classes of singleton classes, let the world know.

4. Every object, classes included, has its own "real class," be it a regular class or a singleton class.

5. Every class, with the exception of BasicObject, has exactly one ancestor—either a superclass or a module. This means you have a single chain of ancestors from any class up to BasicObject.

6. The superclass of the singleton class of an object is the object's class. The superclass of the singleton class of a class is the singleton class of the class's superclass. (Try repeating that three times, fast. Then look back at Figure 8, *Singleton classes and inheritance*, on page 124, and it will all make sense.)

7. When you call a method, Ruby goes "right" in the receiver's real class and then "up" the ancestors chain. That's all there is to know about the way Ruby finds methods.

Any Ruby programmer can stumble on a difficult question about the object model. "Which method in this complicated hierarchy gets called first?" Or maybe, "Can I call this method from that object?" When this happens to you, review the seven rules listed earlier, maybe draw a quick diagram of the object model, and you'll find the answer in no time at all.

Congratulations—you now understand the entire Ruby object model.

### Class Methods Syntaxes

Because class methods are just Singleton Methods that live in the class's singleton class, now you have three different ways to define a class method. They're shown in the following code.